InstructionDialog UI仕様書

1. 概要

1.1. コンポーネントの目的

プロジェクト管理画面用の作業指示送信ダイアログ。実験管理者がタスク詳細（メソッド、試行回数、実験条件）を設定し、作業者に対して複数の指示を送信できます。ドラッグ&ドロップによるタスク順序変更、トランジション・タスク別の指示、連続指示モードなど、高度な機能を提供します。

1.2. 使用場面

- ProjectManagement画面から作業指示を送信する場合
- プロジェクトに作業者を割り当て、タスクを設定する場合
- 複数の指示を一括で送信する場合

1.3. アクセス権限

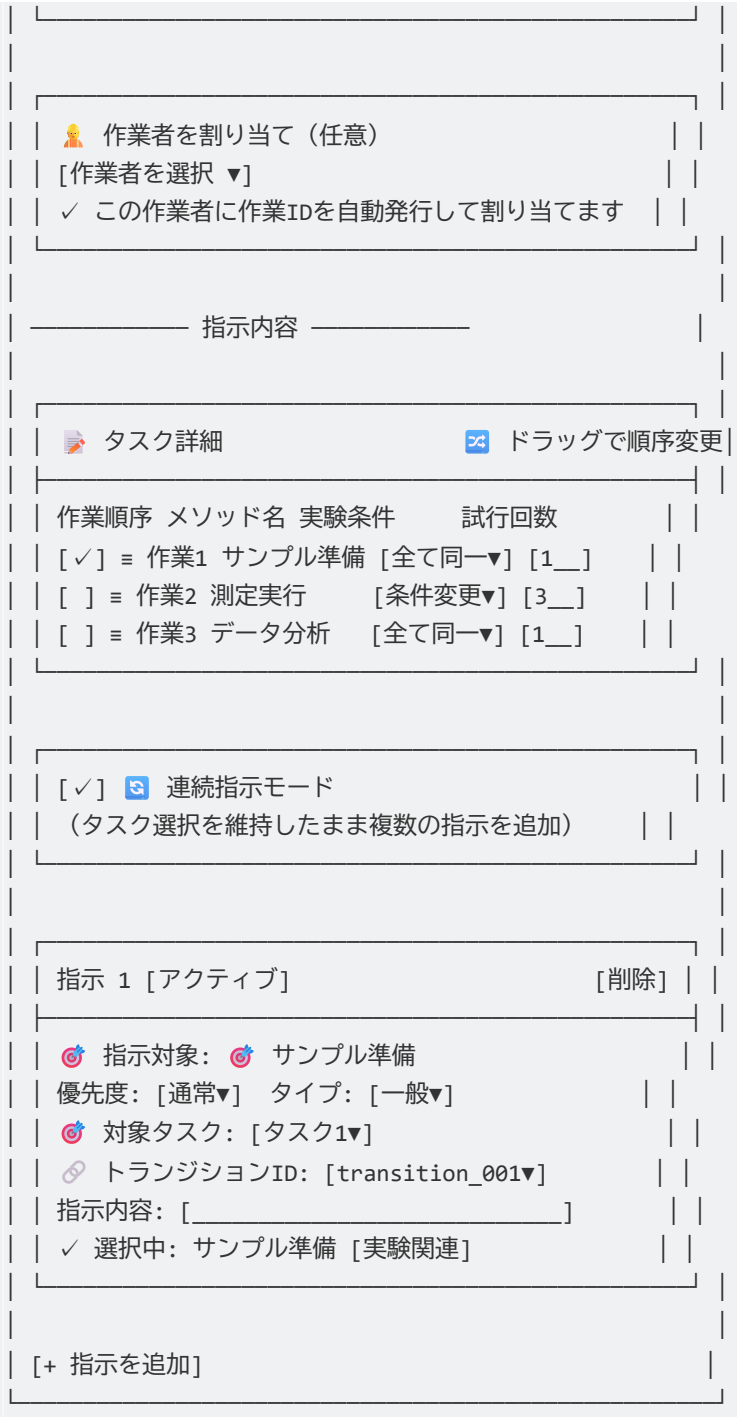
- **対象ロール**: Experiment Admin（実験管理者）
- **必須認証**: JWT認証必須
- **テナント分離**: organization_id によるマルチテナント分離

---

2. 画面構成

2.1. レイアウト構造

```
┌──────────────────────────────────────────────────┐
│ Modal: 作業指示入力              [キャンセル] [完了] │
├──────────────────────────────────────────────────┤
│ ┌──────────────────────────────────────────────┐ │
│ │ ℹ️ プロジェクト情報                          │ │
│ │ プロジェクト名: 材料実験A                    │ │
│ │ 所有者: 佐藤太郎   ステータス: [進行中]      │ │
```



2.2. 主要UI要素

2.2.1. モーダル設定

項目	値
幅	800px
タイトル	「作業指示入力」+ FileTextOutlined

ボディ最大高さ	70vh
スクロール	縦スクロール有効

2.2.2. フッターボタン

ボタン	タイプ	アイコン	動作
キャンセル	default	-	ダイアログを閉じる
指示準備完了	primary	SendOutlined / LoadingOutlined	送信処理実行

2.2.3. プロジェクト情報Alert

表示項目:

- プロジェクト名
- 所有者
- プロジェクトステータス（Tag）

スタイル:

- type: info
- showIcon: true
- icon: InfoCircleOutlined

2.2.4. 作業者選択Card

表示条件: availableWorkers.length > 0

要素	コンポーネント	説明
ラベル	Text strong	「👷 作業者を割り当て（任意）」
選択	Select	allowClear, 幅100%
確認Alert	Alert（成功）	選択時に表示

背景色: colorWarningBg

2.2.5. タスク詳細Card

タイトル:

- 左:「📝 タスク詳細」
- 右:「🔀 ドラッグで順序変更可能」(2件以上の場合)

ヘッダー行:

カラム	幅	説明
チェックボックス	24px	タスク選択
ドラッグハンドル	24px	HolderOutlined
作業順序	80px	「作業順序 N」
メソッド名	flex: 1	メソッド表示名 + 説明
実験条件	180px	Select(全て同一条件 / 条件を変更)
試行回数	100px	Input(number, 1-999)

ドラッグ機能:

- @dnd-kit/coreとsortableを使用
- PointerSensor: 8px移動でドラッグ開始
- verticalListSortingStrategy

選択状態:

- 背景色: colorPrimaryBg
- ボーダー: colorPrimary

2.2.6. 連続指示モードCheckbox

項目	値
ラベル	「🔁 連続指示モード」
説明	「(タスク選択を維持したまま複数の指示を追加)」
デフォルト	true

2.2.7. 指示入力Card

タイトル:

- 左:「指示 N」+ アクティブTag(条件付き)

- 右: 削除ボタン（2件以上の場合）

アクティブ状態:

- 背景色: colorPrimaryBg
- ボーダー: 2px solid colorPrimary
- クリックでアクティブ化可能

入力項目:

項目	コンポーネント	幅	説明
指示対象ラベル	Card	100%	選択中のメソッド名を表示
優先度	Select	flex: 1	高 / 中 / 通常 / 低
タイプ	Select	flex: 1	一般 / セットアップ / 測定 / 安全 / 清掃
対象タスク	Select	flex: 1	タスク番号選択（条件付き）
トランジションID	Select	flex: 1	トランジション選択
指示内容	TextArea	100%	maxLength: 1000, rows: 4, showCount

選択済み情報Alert:

- トランジション選択時: 成功Alert（名前、カテゴリ、説明）
- タスク選択時: 成功Alert（タスク番号）

2.2.8. 指示追加ボタン

項目	値
タイプ	dashed
アイコン	PlusOutlined
テキスト	「指示を追加」
幅	block（100%）

2.3. 画面遷移

- なし（モーダルダイアログ）
- 送信成功後にダイアログを閉じる

3. 機能仕様

3.1. 主要機能の詳細

3.1.1. モーダルオープン時の初期化

処理フロー:

1. useEffect()でopen===trueを検知
2. MAIMLファイルからメソッド情報を取得（fetchMethods）
3. MAIMLファイルからトランジション情報を取得（fetchTransitions）
4. メソッド取得順にタスク詳細を初期化
5. 初期指示を1件作成

依存関係: [open, fileKey]

3.1.2. モーダルクローズ時のリセット

処理フロー:

1. useEffect()でopen===falseを検知
2. 全状態をリセット:
    - instructions: 初期値
    - selectedWorkerId: 空
    - availableTransitions: 空配列
    - availableMethods: 空配列
    - taskDetails: 初期値
    - selectedTaskId: null
    - activeInstructionId: 1
    - continuousInstruction: true

3.1.3. メソッド情報取得（モックデータ）

処理フロー:

1. 500ms遅延でローディングをシミュレート
2. モックのメソッド情報を生成:

```
{
  id: 'method_001',
  name: 'sample_preparation',
  displayName: 'サンプル準備',
  description: '実験サンプルの準備プロセス'
}
```

3. メソッド取得順にタスク詳細を初期化
4. 各タスクにmethodIdを自動設定

TODO: 実際のAPI実装時に置き換え

3.1.4. トランジション情報取得（モックデータ）

処理フロー:

1. 500ms遅延でローディングをシミュレート
2. モックのファイル基本情報を設定
3. モックのトランジション情報を生成:

```
{
  id: 'transition_001',
  displayName: 'サンプル準備',
  category: 'preparation',
  description: '実験サンプルの準備プロセス',
  isExperimentRelevant: true
}
```

エラーハンドリング:

- エラー時はmaimlParseErrorに設定
- message.error()で通知

TODO: 実際のAPI実装時に置き換え

3.1.5. タスク詳細の更新

処理フロー:

1. handleUpdateTaskDetail()を呼び出し
2. 該当タスクのフィールドを更新
3. experimentConditionの場合は数値型に変換（0 | 1）
4. trialCountの場合、taskNumberOptionsも自動更新:

```
const newOptions = Array.from({ length: newTrialCount }, (_, i) => i + 1);
```

5. 試行回数変更時、影響を受ける指示のタスク番号をリセット

バリデーション:

- trialCount: 1-999の範囲

3.1.6. タスク選択/解除（トグル）

処理フロー:

1. handleToggleTaskSelection()を呼び出し
2. 単一選択のトグル動作
3. 選択時: アクティブな指示にmethodIdを設定
4. 解除時: methodIdをクリア

3.1.7. タスク順序の変更（ドラッグ&ドロップ）

処理フロー:

1. DndContextとSortableContextで制御
2. handleDragEnd()でactive.idとover.idを比較
3. arrayMove()で配列を並び替え
4. taskDetailsを更新

制約:

- submitting===trueの場合は無効化

3.1.8. 指示の追加

処理フロー:

1. handleAddInstruction()を呼び出し
2. 新しいIDを生成（prev.length + 1）
3. 連続指示モードONの場合、選択中タスクのmethodIdを設定
4. 新しい指示を配列に追加
5. 新しい指示をアクティブ化
6. setTimeout()でrestoreTaskSelection()を呼び出し

3.1.9. 指示の削除

処理フロー:

1. handleRemoveInstruction()を呼び出し
2. 該当指示をフィルタで除外
3. アクティブな指示が削除された場合:
    - 残っている最初の指示をアクティブ化
    - setTimeout()でrestoreTaskSelection()を呼び出し

制約:

- 最低1件は必要（削除ボタンを非表示）

3.1.10. 指示のアクティブ化

処理フロー:

1. 指示Cardをクリック
2. handleActivateInstruction()を呼び出し
3. activeInstructionIdを更新
4. restoreTaskSelection()でタスク選択状態を復元

3.1.11. 指示の更新

処理フロー:

1. handleUpdateInstruction()を呼び出し
2. 該当指示のフィールドを更新
3. targetType変更時:
    - FILE: transitionIdとtaskNumberをリセット
    - TASK: transitionIdをリセット
    - TRANSITION: taskNumberをリセット

3.1.12. 送信処理

バリデーション:

1. すべてのメソッドが選択されているか
2. 試行回数が1-999の範囲か
3. 少なくとも1つの指示が入力されているか
4. トランジション指示にトランジションIDが設定されているか
5. タスク指示にタスク番号が設定されているか

処理フロー:

1. handleSubmit()を呼び出し
2. バリデーション実行
3. submitting: true
4. instructionDataを構築:

```
{
  instructionId: `instruction_${Date.now()}_${inst.id}`,
  content: inst.content.trim(),
  priority: inst.priority,
  instructionType: inst.type,
  type: inst.type,
  timestamp: new Date().toISOString(),
  targetType: inst.targetType,
  transitionId: inst.transitionId || null,
```

```
    taskNumber: inst.taskNumber
  }
```

5. onSubmit()コールバックを実行
6. message.success()で成功通知
7. onClose()でダイアログを閉じる

エラーハンドリング:

- message.error()で通知
- submitting: false

3.1.13. ダイアログを閉じる

処理フロー:

1. handleClose()を呼び出し
2. submitting===trueの場合:
    - Modal.confirm()で確認
    - 「送信中」「キャンセルしますか？」
3. submitting===falseの場合:
    - 即座にonClose()を実行

3.2. ボタン・操作の動作

操作	動作	状態変更
作業者選択	作業者を設定	selectedWorkerId更新
タスクチェックボックス	タスク選択/解除	selectedTaskId更新、アクティブ指示のmethodId更新
タスクドラッグ	順序変更	taskDetails並び替え
実験条件変更	条件を更新	task.experimentCondition更新
試行回数変更	回数を更新	task.trialCount更新、taskNumberOptions更新
連続指示チェック	モード切替	continuousInstruction更新
指示Card クリック	アクティブ化	activeInstructionId更新、タスク選択復元

指示削除ボタン	指示を削除	instructions更新
優先度/タイプ変更	指示を更新	instruction更新
対象タスク/トランジション選択	対象を設定	instruction更新
指示内容入力	内容を更新	instruction.content更新
指示追加ボタン	指示を追加	instructions追加
キャンセルボタン	ダイアログを閉じる	open: false
指示準備完了ボタン	送信処理	submitting: true → false

3.3. バリデーションルール

項目	ルール	エラーメッセージ
メソッド選択	すべて必須	「すべてのメソッドを選択してください」
試行回数	1-999の範囲	「試行回数は1〜999の範囲で入力してください」
指示内容	最低1件必須	「少なくとも1つの指示を入力してください」
トランジションID	トランジション指示の場合必須	「トランジション指示には対象トランジションを選択してください」
タスク番号	タスク指示の場合必須	「タスク指示には対象タスクを選択してください」

---

4. データ仕様

4.1. Props型定義

```
interface InstructionDialogProps {
  open: boolean;                          // モーダル表示状態
  onClose: () => void;                    // 閉じるコールバック
  onSubmit?: (data: SubmitData) => void;  // 送信コールバック
  fileKey: string;                        // ファイルキー
  managerId: string;                      // 管理者ID
  projectName: string;                    // プロジェクト名
  projectOwner: string;                   // プロジェクト所有者
  projectStatus: string;                  // プロジェクトステータス
  selectedFile?: Record<string, unknown>; // 選択ファイル（未使用）
```

```
  availableWorkers?: WorkerInfo[];          // 利用可能な作業者
}
```

4.2. InstructionData型定義

```
interface InstructionData {
  id: number;                // 指示ID（ローカル）
  content: string;           // 指示内容
  priority: string;          // 優先度
  type: string;              // タイプ
  targetType: string;        // 対象タイプ
  transitionId: string;      // トランジションID
  taskNumber?: number;       // タスク番号
  methodId: string;          // メソッドID
}
```

4.3. TaskDetail型定義

```
interface TaskDetail {
  id: number;                     // タスクID（ローカル）
  methodId: string;               // メソッドID
  experimentCondition: 0 | 1;     // 実験条件（0: 全て同一, 1: 条件変更）
  trialCount: number;             // 試行回数
  taskNumberOptions: number[];    // タスク番号選択肢
}
```

4.4. MethodInfo型定義

```
interface MethodInfo {
  id: string;               // メソッドID
  name: string;             // メソッド名（英語）
  displayName: string;      // 表示名（日本語）
  description?: string;     // 説明
}
```

4.5. TransitionInfo型定義

```
interface TransitionInfo {
  id: string;                    // トランジションID
  displayName: string;           // 表示名
  category: string;              // カテゴリ
  description?: string;          // 説明
  isExperimentRelevant: boolean;   // 実験関連フラグ
}
```

4.6. SubmitInstructionData型定義

```
interface SubmitInstructionData {
  instructionId: string;           // 指示ID（ユニーク）
  content: string;                 // 指示内容
  priority: string;                // 優先度
  instructionType: string;         // 指示タイプ
  type: string;                    // タイプ（同上）
  timestamp: string;               // タイムスタンプ
  targetType: string;              // 対象タイプ
  transitionId: string | null;     // トランジションID
  targetTransitionId?: string;     // 対象トランジションID
  taskNumber?: number;             // タスク番号
}
```

---

5. API連携

5.1. 直接使用するAPI

このコンポーネント自体はAPIを直接呼び出しません（現在はモックデータで動作）。

5.2. 使用予定API一覧

API	説明	実装状態

GET `/api/v1/maiml/methods`	メソッド情報取得	🔴 未実装（モック）
GET `/api/v1/maiml/transitions`	トランジション情報取得	🔴 未実装（モック）
POST `/api/v1/instructions`	作業指示送信	🔴 未実装

5.3. API仕様詳細

5.3.1. メソッド情報取得 🔴

使用予定API:

- GET `/api/v1/maiml/methods` - MAIMLメソッド情報取得 🔴

優先度: ⭐⭐⭐ 高

詳細: 将来的にMAIML編集API仕様書に追加予定

Request:

```
GET /api/v1/maiml/methods?project_id={project_id}
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
X-Organization-Id: {organization_id}
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "methods": [
      {
        "method_id": "method_001",
        "method_name": "sample_preparation",
        "display_name": "サンプル準備",
        "description": "実験サンプルの準備プロセス",
        "order": 1
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-23T08:00:00Z",
    "requestId": "req_abc123"
```

```
  }
}
```

5.3.2. トランジション情報取得 🔴

使用予定API:

- GET `/api/v1/maiml/transitions` - MAIMLトランジション情報取得 🔴

優先度: ⭐⭐⭐ 高

詳細: 将来的にMAIML編集API仕様書に追加予定

Request:

```
GET /api/v1/maiml/transitions?project_id={project_id}
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
X-Organization-Id: {organization_id}
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "transitions": [
      {
        "transition_id": "transition_001",
        "display_name": "サンプル準備",
        "category": "preparation",
        "description": "実験サンプルの準備プロセス",
        "is_experiment_relevant": true
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-23T08:00:00Z",
    "requestId": "req_abc123"
  }
}
```

5.3.3. 作業指示送信 🔴

使用予定API:

- POST `/api/v1/instructions` - 作業指示送信 🔴

優先度: ⭐⭐⭐ 高

詳細: 将来的に進捗確認または指示管理API仕様書に追加予定

Request:

```
POST /api/v1/instructions
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
X-Organization-Id: {organization_id}
Content-Type: application/json

{
  "project_id": "550e8400-e29b-41d4-a716-446655440000",
  "manager_id": "user-uuid-001",
  "assigned_worker_id": "user-uuid-002",
  "instructions": [
    {
      "instruction_id": "instruction_1737595200_1",
      "content": "サンプルを準備してください",
      "priority": "normal",
      "instruction_type": "general",
      "target_type": "TRANSITION",
      "transition_id": "transition_001",
      "task_number": null,
      "timestamp": "2026-01-23T08:00:00Z"
    }
  ],
  "task_details": [
    {
      "method_id": "method_001",
      "experiment_condition": 0,
      "trial_count": 1
    }
  ]
}
```

Response (201 Created):

```
{
  "success": true,
  "data": {
    "instruction_batch_id": "batch_abc123",
    "created_instructions": [
      {
        "instruction_id": "instruction_1737595200_1",
        "status": "pending",
        "created_at": "2026-01-23T08:00:00Z"
      }
    ],
    "assigned_worker": {
      "worker_id": "user-uuid-002",
      "worker_name": "鈴木花子",
      "work_id": "work_abc123"
    }
  },
  "meta": {
    "timestamp": "2026-01-23T08:00:00Z",
    "requestId": "req_abc123"
  }
}
```


5.4. モックデータ使用

現在の実装:

- メソッド情報: 500ms遅延後にモックデータを返す
- トランジション情報: 500ms遅延後にモックデータを返す
- 送信処理: onSubmit()コールバックで親に委譲（実際のAPI呼び出しは親コンポーネントが実装）

---

6. エラーハンドリング

6.1. エラーケース

エラーケース	処理	表示
メソッド取得失敗	message.error()	エラートースト
トランジション取得失敗	maimlParseError設定	エラートースト

バリデーションエラー	message.warning()	警告トースト
送信処理失敗	message.error()	エラートースト

6.2. エラーメッセージ

バリデーションエラー:

- 「すべてのメソッドを選択してください」
- 「試行回数は1〜999の範囲で入力してください」
- 「少なくとも1つの指示を入力してください」
- 「トランジション指示には対象トランジションを選択してください」
- 「タスク指示には対象タスクを選択してください」

送信中の確認:

- タイトル: 「送信中」
- 内容: 「指示データの準備が進行中です。キャンセルしますか？」

---

7. UI/UXポリシー準拠

7.1. Ant Designコンポーネント使用

コンポーネント	使用箇所	props
Modal	ダイアログ本体	title, open, onCancel, width, footer, styles
Space	要素の配置	direction, size, style
Alert	プロジェクト情報、確認	message, description, type, showIcon, icon
Card	セクション	size, title, style, onClick, hoverable
Select	選択入力	value, onChange, disabled, placeholder, options
Input	試行回数入力	type, value, onChange, onBlur, min, max, size
Checkbox	タスク選択、連続指示	checked, onChange, disabled
Radio	将来的に使用予定	-
Button	削除、追加、送信	type, onClick, icon, disabled, block

Tag	ステータス、アクティブ	color
Typography	テキスト表示	Text, Title
Flex	フレックスレイアウト	gap, align, justify
Divider	区切り線	style
Spin	ローディング	tip
TextArea	指示内容入力	value, onChange, rows, showCount, maxLength

7.2. @dnd-kit使用

パッケージ	使用箇所	説明
@dnd-kit/core	DndContext	ドラッグコンテキスト
@dnd-kit/sortable	SortableContext, useSortable	ソート可能リスト
@dnd-kit/utilities	CSS	トランスフォーム変換

7.3. デザイントークン使用

```
import { theme } from 'antd';
const { token } = theme.useToken();

// 使用箇所
padding: token.padding
paddingSM: token.paddingSM
paddingXS: token.paddingXS
paddingXXS: token.paddingXXS
margin: token.margin
marginSM: token.marginSM
marginXS: token.marginXS
marginLG: token.marginLG
borderRadius: token.borderRadius
colorPrimary: token.colorPrimary
colorPrimaryBg: token.colorPrimaryBg
colorBorder: token.colorBorder
colorBgContainer: token.colorBgContainer
```

---

8. 実装ファイル

8.1. ファイルパス

- **メインファイル**: 未実装（仕様書のみ存在）

注記: このコンポーネントは仕様書が作成されていますが、実装ファイルは `frontend/src/components/progress/` 配下に存在しません。実装が必要な場合は、この仕様書を参照して作成してください。

---

9. 変更履歴

日付	変更内容	担当者
2026-03-11	実装状況確認（実装ファイルなし、仕様書のみ存在）	Cline